

Big Data and Automated Content Analysis
(6 ECTS)

Course Manual

dr. Anne Kroon
Graduate School of Communication
University of Amsterdam
a.c.kroon@uva.nl

Academic Year 2025/26
Semester 1, block 2

Chapter 1

About this course

This course manual contains general information, guidelines, rules and schedules for the Research Master course Big Data & Automated Content Analysis Part I (6 ECTS). Please make sure you read it carefully, as it contains information regarding assignments, deadlines and grading.

1.1 Course description

“Big data” refers to data that are more voluminous, but often also more unstructured and dynamic, than traditionally the case. In Communication Science and the Social Sciences more broadly, this in particular concerns research that draws on Internet-based data sources such as social media, large digital archives, and public comments to news and products. This emerging field of studies is also called *Computational Social Science* (Lazer et al., 2009) or, narrowed down to the analysis of communication, *Computational Communication Science* (Shah, Cappella, & Neuman, 2015).

The course will provide insights in the concepts, challenges and opportunities associated with data so large that traditional research methods (like manual coding) cannot be applied any more and traditional inferential statistics start to lose their meaning. Participants are introduced to strategies and techniques for capturing and analyzing digital data in communication contexts. We will focus on (a) data harvesting, storage, and preprocessing and (b) computer-aided content analysis, including natural language processing (NLP) and computational social science approaches. In particular, we will use advanced machine learning approaches and models like word embeddings.

To participate in this course, students are ex-

pected to be interested in learning how to write own programs in Python. Some basic understanding of programming languages is helpful, but not necessary to enter the course. Students without such knowledge are encouraged to follow one of the many (free) online introductions to Python to prepare.

1.2 Goals

Upon completion of this course, the following goals are reached:

3. Knowledge and Understanding: Have in-depth knowledge and a thorough understanding of advanced research designs and methods.

3.1. Have in-depth knowledge and a thorough understanding of advanced research designs and methods, including their value and limitations.

3.2. Have in-depth knowledge and a thorough understanding of advanced techniques for data analysis.

A Students can explain *fundamental research designs* and *commonly employed methods* in existing research articles on Big Data and automated content analysis.

B Students can on their own and in own words critically discuss the pros and cons of *fundamental* research designs and methods employed in existing research articles on Big Data and automated content analysis; they can, based on this, give a critical evaluation of the methods and, where relevant, give advice to improve the study in question.

C Students are able to identify some *basic* techniques from the field of computer science and computer linguistics that are applicable to research in communication science; they can

explain the principle of some traditional approaches to text analysis, namely simple rule-based techniques and basic methods of unsupervised and supervised machine learning.

4. Skills and abilities: Are able, independently and on their own, to set up, conduct, report and interpret advanced academic research.

4.1 Are able to formulate research questions and hypotheses for advanced empirical studies

4.2 Are able to develop a research plan, choose appropriate and suitable research designs and methods for advanced empirical studies, and justify the underlying choices.

4.3 Are able to assess the validity and reliability of advanced empirical research, and to judge the scientific and professional value of findings from advanced empirical research.

4.4 Are able to apply advanced empirical research methods.

D Students can on their own formulate a research question and hypotheses for own empirical research in the domain of Big Data.

E Students can on their own chose, execute and report on *fundamental* research methods in the domain of Big Data and automatic content analysis.

F Students know how to collect data with APIs or read in existing data files; they know how to analyze these data with fundamental automated techniques and to this end, they have basic knowledge of the programming language Python and know how to use fundamental Python-modules for communication science research.

6. Academic attitudes

6.1 Regularly asses their own assumptions, strengths and weaknesses critically.

6.2 Accept that scientific knowledge is always 'work in progress' and that something regarded as 'true' may be proven to be false, and vice-versa.

6.3 Are keen to acquire new knowledge, skills and abilities.

6.4 Are willing to share and discuss arguments, results and conclusions, including submitting one's own work to peer review.

6.5 Are convinced that academic debates should not be conducted on the basis of rhetorical qualities but that arguments must be considered and conclusions drawn on the basis of empirical results and valid criticism.

G Students can critically discuss strong and weak points of their own research using fundamental techniques from the field of Big Data and Automated Content Analysis, and suggest improvements.

H Students participate actively: reading the literature carefully and on time, completing assignments carefully and on time, active participation in discussions, and giving feedback on the work of fellow students give evidence of this.

1.3 Help with practical matters

While making your first steps with programming in Python, you will probably have a lot of questions. Nevertheless, <https://google.com> and <https://stackoverflow.com> should be your first points of contact. After all, that's how we solve our problems as well...

It is also very much encouraged to try to solve problems together with your classmates. A successful technique can be *pair programming*, where two colleagues sit behind one screen (for an explanation, see https://en.wikipedia.org/wiki/Pair_programming).

If you really cannot find a solution on your own, you can pose such questions during the Thursday lab sessions. However, keep the remark in Section 2.5 in mind!

Chapter 2

Rules, assignments, and grading

The final grade of this course will be composed of the grade of one mid-term take home exam (30%) and one individual project (70%).

Mid-term take-home exam (30%)

In the mid-term take-home exam, students will show their understanding of the literature and prove they have gained new insights during the lecture/seminar meetings. They will be asked to critically assess various approaches to Big Data analysis and make own suggestions for research. Additionally, they need to (partly) write the code to accomplish this.

Grading criteria are communicated to the students together with the assignment, but in general are: For literature-related tasks in the exam:

- usage of specific examples from the literature;
- critique of different approaches;
- nameing of pro's, con's, potential pitfalls, and alternatives;
- giving practical advice and guidance.

For programming-related tasks in the exam:

- correctness, efficiency, and style of the code
- correctness, completeness, and usefulness of analyses applied.

For conceptual and planning-related tasks:

- feasibility
- level of specificity

- explanation and argumentation why a specific approach is chosen
- creativity.

Final individual project (70%)

The final individual project typically consists of the following elements, which all contribute to the final grade:

- introduction including references to relevant (course) literature, an overarching research question plus subquestions and/or hypotheses (1–2 pages);
- an overview of the analytic strategy, referring to relevant methods learned in this course;
- carefully collected and relevant dataset of non-trivial size; here, using APIs, or combining existing datasets.
- a set of scripts for collecting, preprocessing, and analyzing the data using fundamental techniques discussed in the course. The scripts should be well-documented and tailored to the specific needs of the own project;
- output files;
- a well-substantiated conclusion with an answer to the RQ and directions for future research.

Depending on the choosen topic, the student will have to apply some of the techniques covered in the course. The assignment needs to present an explorative description of the dataset, combined with some of the fundamental techniques discussed during the course. During the lab sessions, as well as

individual consultation, students may discuss the scope of the projects with the lecturer, the requirements that the specific project suggested by the student needs to fulfill, and the extent to which the different methods that the student plans to use will contribute to the final grade.

Grading and 2nd try

Students have to get a pass (5.5 or higher) for both mid-term take-home exam and the individual project. If the grade of one of these is lower, an improved version can be handed in within one week after the grade is communicated to the student. If the improved version still is graded lower than 5.5, the course cannot be completed. Improved versions of the final individual project cannot be graded higher than 6.0.

2.1 Presence and participation

Attendance is compulsory. Missing more than two meetings – for whatever reason – means the course cannot be completed.

Next to attending the meetings, students are also required to prepare the assigned literature and to continue working on the programming tasks after the lab sessions. To successfully finish the course, attending the lab sessions is not enough, but has to go hand-in-hand with continuous self-study.

2.2 Staying informed

It is your responsibility to check the means of communications used for this course (i.e., your email account, but – if applicable – also e-learning platforms or any other tool that the lecturer decides to use) on a regular basis, which in most cases means daily.

2.3 Plagiarism & fraud

Plagiarism is a serious academic violation. Cases in which students use material such as online sources or any other sources in their written work and present this material as their own original work

without citation/referencing, and thus conduct plagiarism, will be reported to the Examencommissie of the Department of Communication without any further negotiation. If the committee comes to the conclusion that a student has indeed committed plagiarism the course cannot be completed. General UvA regulations about fraud and plagiarism apply, see also <https://student.uva.nl/en/content/az/plagiarism-and-fraud/plagiarism-and-fraud.html>.

2.3.1 Attributing code

Plagiarism rules apply to code in the same way as they apply to text. Just like how your writing often builds on other scholars (who you cite), your code often builds on what others have done or are inspired by solutions of others, for instance in communities like Stackoverflow. Analogous to plagiarism regulations in normal writing, also in your code, it needs to be transparent *what is from you and what is not*. We don't really have something like APA references for code, but you can simply use `#comments` for that purpose, such as: `# The following function is copied from https://stackoverflow.com/XXXXX/XXXXX for (almost) literal copy-pasting, or # The code in this cell is inspired by https://...; I modified Y and Z.`

Make sure that there is no doubt about what is yours and what is from someone else!

2.3.2 Use of AI tools

We encourage you to develop your own analytical and research skills throughout this course.

- Do not use AI tools to generate, write, or substantially complete any assignments or projects that you submit for assessment. Doing so will be treated as academic misconduct.
- You may use AI tools responsibly to clarify concepts, review methodologies, or help debug analyses that you have conducted yourself. The final work you submit must be authored by you and fully understood by you.
- Prefer using UvA AI Chat for privacy and data protection. Be cautious with third-party tools and do not share sensitive or personal data.

- If you use AI, disclose and cite the tool and briefly describe how you used it. Normal plagiarism and fraud rules apply.

2.3.3 Working together

The take-home exams as well as the final project are *individual exams*. Collaboration with fellow students is thus considered academic fraud and will be reported and handled accordingly.

In some instances, some overlap between final projects may be unavoidable, for instance when relying on very similar data sources. If in doubt, always discuss with me in advance to avoid problems after you handed in! It is almost always possible to find a solution, if discussed in advance.

*In contrast to these graded assignments, for all **ungraded** assignments and exercises, collaboration is explicitly encouraged. Experience with this course over the last decade has shown that those students who met regularly with classmates to go over the week's code examples and exercises together enjoyed the course most, and also performed best.*

2.4 Deadlines and handing in

Per assignment, the lecturer will specify whether it has to be handed in via Canvas or via Filesender. If no specific instructions are given, or if there is any issue with submitting an assignment through Canvas, use Filesender. Please send all assignments and papers as a PDF file (and do not use formats like .docx) to ensure that it can be read and is displayed the same way on any device. Multiple files should be compressed and handed in as one .zip file or .tar.gz file. Do not email assignments directly but send them via <https://filesender.surf.nl/> to my mail-address. This way, you can also transfer huge files.

Final papers and take-home exams that are not handed in on time, will be not be graded and receive the grade 1. This rule also applies for any other assignment that might be given. The deadline is only met when the all files are submitted.

2.5 Your computer and operating system: your responsibility

Python is a language, not a program, and there are many ways of running Python code.

It is really hard to help with stuff related to your computer, your operating system, your keyboard etc. Things may work slightly different on Windows compared to MacOS or Linux; it's your responsibility to take care of these things. For instance, a downloaded file may be put under `/home/damian/Downloads` on Linux, under `/Users/damian/Downloads` on MacOS, and under `c:\\Users \\Damian\\Downloads` on Windows – or maybe something very different. You are expected to have a sufficiently deep understanding of your own computer to deal with these idiosyncrasies yourself.

Therefore you need to make sure that the following things apply. Otherwise, you need to fix it – we won't have room for that during the course. So, do this **before the course starts**.

- You need to have a recent version of a Python interpreter installed, and you need to be able to run Jupyter Notebooks. Chapter 1 of van Atteveldt, Trilling, and Arcila Calderón (2022) gives instructions (you can skip the part about R, though). You can either use Python natively or use Anaconda (see Chapter 1).
- You need to be able to install third-party packages (see Chapter 1). Test this by checking whether you can install the package `shifterator` and then load it with `import shifterator` without getting an error.
- You need to make sure that your keyboard produces straight quotes `"` rather than typographical ones when typing. Test that by making sure that you can run `print("hello world")` in a Jupyter notebook.
- Create a folder for this course at an easy to remember place, such

as `/home/damian/bdaca` (Linux), `/Users/damian/bdaca` (MacOS), or `c://Users//damian//bdaca` (Windows). You could potentially also use `c://Users//damian//Desktop//bdaca` (Win) or `/Users/damian/Desktop/bdaca` (Mac) or similar. Make sure that you can locate files in that folder in Jupyter Notebook. And remember/write down its name! Preferably, don't use spaces (can lead to confusion), and Windows users, replace `\` by either a double `\\` or by forward slashes `/`.

If – even after trying – you do not succeed in any of these steps, ask your classmates. If that also does not help, contact me as soon as possible.

Chapter 3

Schedule and Literature

The following schedule gives an overview of the topics covered each week, the obligatory literature that has to be studied each week, and other tasks the students have to complete in preparation of the class. In particular, the schedule shows which chapter of van Atteveldt et al. (2022) will be dealt with. Note that some basic chapters that explain how to install the software we are going to use have to be read before the course starts.

Next to the obligatory literature, the following books provide the interested student with more and deeper information. They are intended for the advanced reader and might be useful for final individual projects, but are by no means required literature. Bear in mind, though, that you may encounter slightly outdated examples (e.g., Python 2, now-defunct APIs etc.).

- McKinney, 2012: A lot of examples for data analysis in Python. A HTML-based version of the book can be viewed for free at <https://wesmckinney.com/book/>.
- VanderPlas, 2016: A more recent book on numpy, pandas, scikit-learn and more. It can also be read online for free on <https://jakevdp.github.io/PythonDataScienceHandbook/>. The contents are available as Jupyter Notebooks as well, see <https://github.com/jakevdp/PythonDataScienceHandbook>.
- The pandas cookbook by Julia Evans, a collection of notebooks on github: <https://github.com/jvns/pandas-cookbook>.
- Hovy, 2020: A thin book on bottom-up text analysis in Python with both a bit more math

background and ready-to-use Python code implementations.

- Salganik, 2017: Not a book on Python, but on research methods in the digital age. Very readable, and a lots of inspiration and background about techniques covered in our course.

Before the course starts: Prepare your computer.

✓ CHAPTER 1: INTRODUCTION

Make sure that you have a working Python environment installed on your computer. You cannot start the course if you have not done so.

Week 1: Programming for Computational (Communication|Social) Scientists

Monday, 27–10. Lecture.

✓ KITCHIN (2014)

✓ HILBERT ET AL. (2019)

We discuss what Big Data and Computational (Social|Communication) Science are. We talk about challenges and opportunities as well as the implications for the social sciences in general and communication science in particular. We also pay attention to the tools used in CSS, in particular to the use of Python.

Additionally, the journal *Communication Methods and Measures* had a special issue (volume 12, issue 2–3) about Computational Communication Science. Read at least the editorial (van At-

teveldt & Peng, 2018), but preferably, also some of the articles (you can also do that later in the course).

Towards the end of the lecture, we will make first contact with writing code.

Thursday, 30–10. Lecture with exercises.

✓ CHAPTER 3: PROGRAMMING CONCEPTS FOR DATA ANALYSIS

✓ CHAPTER 4: HOW TO WRITE CODE

You will get a very gentle introduction to computer programming. During the lecture, you are encouraged to follow the examples on your own laptop. We will do our first real steps in Python and do some exercises to get the feeling with writing code.

Week 2: From files and APIs to lists, dictionaries, or dataframes

We talk about file formats such as `csv` and `json`; about encodings; about reading these formats into basic Python structures such as dictionaries and lists as opposed to reading them into dataframes; and about retrieving such data from local files, as parts of packages, and via an API.

Monday, 03–11. Lecture.

✓ CHAPTER 5: FROM FILE TO DATAFRAME AND BACK

✓ FREELON (2018-10)

A conceptual overview of different file formats and data sources, and some practical guidance on how to handle such data in basic Python and in Pandas.

Thursday, 06–11. Lab session.

We will exercise with the data structures we learned in Week 1, as well as with different file formats.

Week 3: Data wrangling and exploratory data analysis

Of course, you don't need Python to do statistics. Whether it's R, Stata, or SPSS – you probably already have a tool that you are comfortable with. But you also do not want to switch to a different environment just for getting a correlation. And you definitely don't want to do advanced data wrangling in SPSS. . . This week, we will discuss different ways of organizing your data (e.g., long vs wide formats) as well as how to do conventional statistical tests and simple plots in Python.

Monday, 10–11. Lecture.

✓ CHAPTER 6: DATA WRANGLING

✓ CHAPTER 7.1. SIMPLE EXPLORATORY DATA ANALYSIS

✓ CHAPTER 7.2. VISUALIZING DATA

We will learn how to get your data in the right shape and how to get a first understanding of your data, using exploratory analysis and visualization techniques. We will cover data wrangling with pandas: converting between wide and long formats (melting and pivoting), aggregating data, joining datasets, and so on.

Thursday, 13–11. Lab session and short lecture.

We will apply the techniques discussed during the previous lectures to multiple datasets. In addition, we will start exploring the topic of next week: processing textual data. This short lecture will introduce key ideas about how text can be represented and analyzed using computational methods. We will discuss different approaches to representing textual data (such as tokenization and Bag-of-Words models) and experiment with some simple examples in Python.

Week 4: Processing textual data

In this week, we will dive into how to work with textual data. How is text represented, how can we

process it, and how can we extract useful information from it? Because text as written by humans is often quite messy, we will explore ways to represent it in a cleaner and more structured way. We will introduce the Bag-of-Words (BOW) representation and demonstrate several ways of transforming text into matrices that can be analyzed quantitatively.

Monday, 17–11. Short lecture and lab session.

- ✓ CHAPTER 9: PROCESSING TEXT
- ✓ CHAPTER 10: TEXT AS DATA
- ✓ CHAPTER 11, SECTIONS 11.1–11.3: AUTOMATIC ANALYSIS OF TEXT

Additional recommended background reading on stopwords: Nothman, Qin, and Yurchak (2018).

This session combines a short lecture with a hands-on exercise. We will further explore techniques such as lemmatization, stopword removal, n-grams, word counts and word co-occurrences, and regular expressions. Building on Thursday’s introduction, we will implement these methods in Python and continue developing our first automated content analysis script, combining our knowledge of data structures and programming concepts with introductory text-processing techniques.

Thursday, 20–11. No class; time to work on your take-home exam.

You can use this time to continue working on your take-home exam and apply the concepts discussed in the lectures.

Week 5: Unsupervised approaches to text analysis

In this week, we will make the transition from traditional statistical models to modern unsupervised machine learning techniques for text analysis. We will discuss how these approaches relate to earlier methods such as topic modeling and how recent advances in transformer-based language models have reshaped exploratory text analysis.

Monday, 24–11. Lecture: From topic models to embeddings.

- ✓ CHAPTER 11.5: UNSUPERVISED TEXT ANALYSIS — TOPIC MODELING AND BEYOND
- ✓ KROON, WELBERS, TRILLING, AND VAN ATTEVELDT (2024)
- ✓ MAIER ET AL. (2018)

In this lecture, we will examine the logic of unsupervised text modeling and contrast traditional approaches such as LDA with modern embedding-based techniques. We will introduce the concept of BERTopic as an example of how transformer models can uncover themes in large text collections without manual coding.

Thursday, 27–11. Lab session

In this lab session, we will apply BERTopic to textual data, experimenting with how clustering emerges from contextual embeddings. We will compare its output to simpler unsupervised approaches and discuss strengths, limitations, and interpretability. We will also provide guidance on evaluating unsupervised models and integrating them into computational communication research.

Week 6: Supervised approaches to text analysis

During this week, we will discuss the basics of machine learning. You will be introduced to scikit-learn (Pedregosa et al., 2011), one of the most well-known machine learning libraries. We do not have the time to discuss machine learning techniques in depth. Rather, a practical and hands-on introduction is provided.

Monday, 01–12. Lecture

- ✓ CHAPTER 8: STATISTICAL MODELING AND SUPERVISED MACHINE LEARNING
- ✗ (YOU CAN SKIP 8.4 DEEP LEARNING)
- ✓ BOUMANS AND TRILLING (2016)

We will discuss the basics of supervised machine learning, and how its performance can be evaluated.

Thursday, 04–12. Lab session

Exercises with scikit-learn.

Week 7: Wrapping up

Monday, 08–12. Presentations of Final Projects

During this meeting, everybody will shortly pitch their final projects. This way, you will be able to receive some feedback on your plans.

Thursday, 11–12. Individual consultation

During this meeting, you can consult with the lecturer about questions regarding the final project.

Final project

Deadline for handing in: Wednesday, 17–12, 23.59.

Literature

- Boumans, J. W., & Trilling, D. (2016). Taking stock of the toolkit: An overview of relevant automated content analysis approaches and techniques for digital journalism scholars. *Digital Journalism*, 4(1), 8–23. doi: 10.1080/21670811.2015.1096598
- Freelon, D. (2018-10). Computational research in the post-API age. *Political Communication*, 35(4), 665–668. Retrieved from <https://doi.org/10.1080/10584609.2018.1477506> doi: 10.1080/10584609.2018.1477506
- Hilbert, M., Barnett, G., Blumenstock, J., Contractor, N., Diesner, J., Frey, S., ... Zhu, J. J. H. (2019). Computational Communication Science : A Methodological Catalyzer for a Maturing Discipline. *International Journal of Communication*, 13, 3912–3934.
- Hovy, D. (2020). *Text analysis in Python for social scientists: Discovery and exploration*. Cambridge University Press. doi: 10.1017/9781108873352
- Kitchin, R. (2014). Big Data, new epistemologies and paradigm shifts. *Big Data & Society*, 1(1), 1–12. doi: 10.1177/2053951714528481
- Kroon, A. C., Welbers, K., Trilling, D., & Van Atteveldt, W. (2024, April). Advancing automated content analysis for a new era of media effects research: The key role of transfer learning. *Communication Methods and Measures*, 18(2), 142–162. Retrieved 2024-09-20, from <https://www.tandfonline.com/doi/full/10.1080/19312458.2023.2261372> doi: 10.1080/19312458.2023.2261372
- Lazer, D., Pentland, A., Adamic, L., Aral, S., Barabási, A.-L., Brewer, D., ... van Alstyne, M. (2009). Computational social science. *Science*, 323, 721–723. doi: 10.1126/science.1167742
- Maier, D., Waldherr, A., Miltner, P., Wiedemann, G., Niekler, A., Keinert, A., ... Adam, S. (2018). Applying LDA topic modeling in communication research: Toward a valid and reliable methodology. *Communication Methods and Measures*, 12(2-3), 93–118. doi: 10.1080/19312458.2018.1430754
- McKinney, W. (2012). *Python for data analysis*. O'Reilly.
- Nothman, J., Qin, H., & Yurchak, R. (2018). Stop Word Lists in Free Open-source Software Packages. In *Proceedings of workshop for nlp open source software (nlp-oss)* (pp. 7–12). Association for Computational Linguistics. doi: 10.18653/v1/W18-2502
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Salganik, M. J. (2017). *Bit by bit: Social research in the digital age*. Princeton University Press.
- Shah, D. V., Cappella, J. N., & Neuman, W. R. (2015). Big Data, digital media, and computational social science: Possibilities and perils. *The ANNALS of the American Academy of Political and Social Science*, 659(1), 6–13. doi: 10.1177/0002716215572084
- van Atteveldt, W., Trilling, D., & Arcila Calderón, C. (2022). *Computational analysis of communication: A practical introduction to the analysis of texts, networks, and images with code examples in python and r*. Wiley.
- van Atteveldt, W., & Peng, T. Q. (2018). When Communication Meets Computation: Opportunities, Challenges, and Pitfalls in Computational Communication Science. *Communication Methods and Measures*, 12(2-3), 81–92. doi: 10.1080/19312458.2018.1458084
- VanderPlas, J. (2016). *Python data science handbook: Essential tools for working with data*. O'Reilly.